

SE3040 - Application Frameworks

Deployment Report – SwapNest

1. Introduction

This report describes the deployment process of the SwapNest full-stack application. SwapNest is a MERN-based platform designed to facilitate curated item exchanges, promoting the circular economy with dedicated portals for standard users and volunteers. The system utilizes a monorepo structure containing a React (Vite) frontend, a Node.js (Express) backend, and a MongoDB database.

2. Deployment Architecture

The application relies on a cloud-based infrastructure, managed from a single GitHub repository using root-directory targeting:

- **Frontend:** Hosted on Render (Static Site)
- **Backend:** Hosted on Render (Web Service)
- **Database:** MongoDB Atlas (Cloud Database)

The frontend communicates with the backend via RESTful API calls, and the backend interacts securely with the MongoDB database cluster.

3. Backend Deployment

3.1 Platform Used

Render (Web Service)

3.2 Deployment Steps

1. Pushed the completed source code to the GitHub repository.
2. Logged into the Render dashboard and created a new **Web Service**.
3. Connected the SwapNest GitHub repository.
4. Set the **Root Directory** explicitly to backend to isolate the server code.
5. Configured the build and execution commands:
 - **Build Command:** npm install
 - **Start Command:** node server.js
6. Whitelisted external IP addresses (0.0.0.0/0) within the MongoDB Atlas network access

settings.

7. Added required environment variables in the Render dashboard and deployed the service.

3.3 Environment Variables

- PORT
- MONGO_URI
- CLOUDINARY_CLOUD_NAME
- CLOUDINARY_API_KEY
- CLOUDINARY_API_SECRET
- JWT_SECRET
- RESEND_API_KEY

3.4 Live API URL

<https://swapnest-api.onrender.com>

4. Frontend Deployment

4.1 Platform Used

Render (Static Site)

4.2 Deployment Steps

1. Updated the Axios baseURL in the frontend code to point to the live Render API.
2. Created a new **Static Site** in the Render dashboard and connected the repository.
3. Set the **Root Directory** explicitly to frontend.
4. Configured build settings for the Vite React application:
 - **Build Command:** npm install && npm run build
 - **Publish Directory:** dist
5. Configured routing rules to support React Router's client-side navigation.
6. Deployed the frontend application.

4.3 Environment Variables

- VITE_API_URL

4.4 Live Application URL

<https://swapnest-client.onrender.com/>

5. Integration Between Frontend and Backend

The frontend utilizes **Axios** to execute HTTP requests to the backend API. To ensure proper routing, the base URL is configured with an `/api` suffix (<https://swapnest-api.onrender.com/api>). Global interceptors are used to automatically attach JWT tokens to secure requests. Additionally, the backend CORS middleware is configured to accept cross-origin requests specifically from the deployed frontend domain.

6. Challenges Faced and Solutions

Issue 1: 404 Errors on Page Refresh (Client-Side Routing)

- **Description:** Refreshing any page other than the root (e.g., `/dashboard`) resulted in a 404 Not Found error because the server was looking for physical HTML files that didn't exist in the Single Page Application.
- **Solution:** Configured a Rewrite rule in the Render frontend dashboard (Source: `/*`, Destination: `/index.html`, Action: Rewrite) to ensure React Router intercepts all incoming navigation requests.

Issue 2: Google Authentication Blocked Post-Deployment

- **Description:** Attempting to log in via Google returned an "Access blocked: This app's request is invalid" error because the requests originated from the new Render URL instead of localhost.
- **Solution:** Updated the OAuth 2.0 Client IDs within the Google Cloud Console, adding <https://swapnest-client.onrender.com> to the list of Authorized JavaScript origins.

Issue 3: API Endpoint Not Found (404) During Login

- **Description:** The frontend successfully reached the server, but requests to `/users/login` failed because the backend routes were mounted with an `/api` prefix in `server.js`.
- **Solution:** Appended `/api` to the end of the Axios baseURL in the frontend `api.js` file, ensuring all requests matched the backend's expected `/api/users/login` path.

7. Deployment Evidence

- Backend deployment platform and setup steps.

Waruna-dev / SwapNest master
https://swapnest-api.onrender.com

ⓘ Your free instance will spin down with inactivity, which can delay requests by 50 seconds or more. [Upgrade now](#)

Filter events 31

✓ Deploy live for [139870b](#): Merge pull request #90 from Waruna-dev/Waruna-dev-patch-1 Update frontend URL for password reset
April 11, 2026 at 9:17 PM

Deploy started for [139870b](#): Merge pull request #90 from Waruna-dev/Waruna-dev-patch-1 Update frontend URL for password reset
New commit via Auto-Deploy
April 11, 2026 at 9:16 PM

✓ Deploy live for [592dd6b](#): Update password input fields to include visibility toggle and rename "Forgot?" link to "Forgot Pas...
April 11, 2026 at 8:07 PM [Rollback](#)

- Frontend deployment platform and setup steps

Waruna-dev / SwapNest master
https://swapnest-client.onrender.com

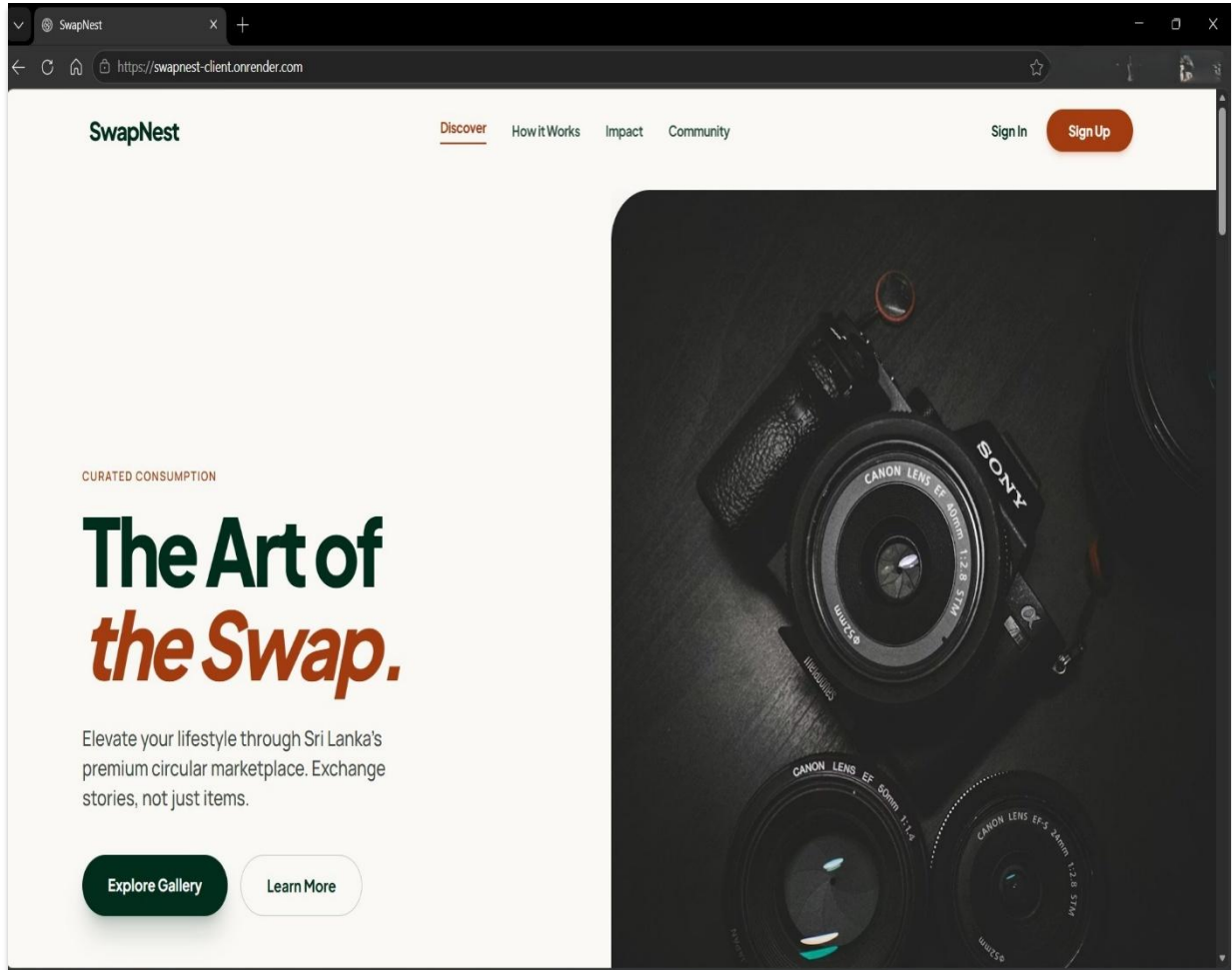
Filter events 28

✓ Deploy live for [5676736](#): Merge pull request #89 from Waruna-dev/Waruna-dev-patch-1 Update API base URL to include '/api' path
April 11, 2026 at 8:32 PM

Deploy started for [5676736](#): Merge pull request #89 from Waruna-dev/Waruna-dev-patch-1 Update API base URL to include '/api' path
New commit via Auto-Deploy
April 11, 2026 at 8:31 PM

✓ Deploy live for [c0532d0](#): Merge pull request #88 from Waruna-dev/Waruna-dev-patch-1 Update API base URL to product...
April 11, 2026 at 8:22 PM [Rollback](#)

- Screenshots of successful deployment.



8. Conclusion

The SwapNest application was successfully deployed using Render's infrastructure, accommodating the monorepo structure by strictly defining root directories. The system is fully functional and accessible online, supporting key operational features such as standard and volunteer authentication, secure password resets, Google OAuth integration, and full database connectivity.